

VIPP 201A — Final Presentation

Speaker Script

Prediction of Driver Aggressiveness Level via Driving Behavior

Hadi Al Shmaissani · Spring 2026 · Supervisor: Prof. Naseem Daher

Target run-time: 15 minutes · 22 slides · ~145 words/minute pace

How to use this script

Each section corresponds to one slide. The boxed cue at the top is a quick reminder of what's on screen so you can deliver the line that matches the visual the audience is reading. The timing budget is approximate — some slides will run 5–10 seconds long or short; the buffer at the end of each major section gives room to recover. Italicized phrases are emphasis points where I usually slow down or pause briefly. Anything in square brackets is a stage direction, not text to read.

Total target: 14 minutes 30 seconds of spoken material plus 30 seconds of buffer for transitions and the pause before Q&A.

Slide 1 · Title · 25s

[visual cue] Title slide showing project name + author + supervisor.

Good afternoon. My name is Hadi Al Shmaissani, and over the next fifteen minutes I'd like to walk you through the spring-term work on a project I've been doing with Professor Daher: predicting driver aggressiveness from driving behavior.

This started as a closed-form index. By the end of the term it's a sensor-noise-validated, multi-environment, ROS-integrated system. The journey getting there is the talk.

Slide 2 · Agenda · 25s

[visual cue] Six numbered cards: Recap, New Architecture, Diagnostics + Fix, SUMO, Friction & ROS2, Sensor-Noise Validation.

Here's the route. I'll start with a quick recap of where the midterm left things, then the new agent architecture, the diagnostic work that surfaced a hidden bias, the population-scale SUMO experiments, the friction-aware regime and the ROS-2 wrapper, and finally the eight-day sensor-noise validation that closed the term.

Slide 3 · Motivation — Why predict driver aggressiveness? · 50s

[visual cue] Two-column compare: 'CURRENT AV STACK' vs 'OUR APPROACH'.

The motivating problem. Today's autonomous-vehicle stacks treat surrounding traffic as moving obstacles with fixed safety buffers — TTC thresholds, headway minimums, that sort of thing.

The trouble is that abstraction throws away most of the information about driver intent. Two cars at the same position and the same velocity can be doing completely different things: one is steady, the other is about to brake hard, lane-change, or close a gap aggressively. The current stack treats them as the same.

Our approach replaces the position-and-velocity tracker with a latent variable — the Aggressiveness Index, or AI — that's computed in closed form from kinematic telemetry, varies smoothly across regimes, and produces a topic that downstream planners can subscribe to. The whole point is to be predictive, not reactive.

Slide 4 · Recap — Where the midterm left off · 50s

[visual cue] Three cards: AI Index formula · Q-learner · Open questions. Bottom strip with Daher's feedback.

The first term established the closed-form index. On the highway, for instance, it's roughly 0.30 times normalized speed plus 0.10 times normalized acceleration plus 0.45 times normalized proximity plus 0.15 times normalized waviness. Weighted sum, scaled to a zero-to-hundred score.

The first term also produced a discrete Q-learner that mapped AI scores to behavioral categories — Conservative, Normal, Aggressive — and reported clean accuracy on highway-env data.

Two questions were left open. Was the Q-learner actually learning, or memorizing? Could the index extend to population-scale traffic? And — added by Professor Daher in the January feedback — the accuracy curve looked tautological, because labels and score come from the same threshold rule. *That feedback shaped most of this term's work.*

Slide 5 • Term-2 contributions — Four pieces, end-to-end · 30s

[visual cue] Dark slide with four numbered tiles.

Four pieces of work, end-to-end. New agent architecture in the upper-left. Diagnostics and a fix for the conservative-bias problem in the upper-right. Population-scale SUMO experiments in the lower-left. And the May second-to-ninth sensor-noise validation in the lower-right. I'll walk through each one.

Slide 6 • From a Q-learner to a weight predictor · 50s

[visual cue] Side-by-side: Term 1 (Q-learner) vs Term 2 (DynamicWeightAgent).

First piece — the new agent. The old Q-learner mapped AI scores to categories. But the labels themselves come from the same formula that produces the score, which means the agent was being graded against an answer key derived from its own input. Tautology.

The DynamicWeightAgent reposes the task. It doesn't classify. It predicts the AI weights themselves, conditioned on environmental context. So instead of saying "AI score 72 means Aggressive," it says "on this stretch of road, with this traffic density and this friction, the right weight on proximity is 0.45, the right weight on waviness is 0.15."

The training loss is a KL term against per-regime targets plus an RL term that maximizes one-minus-AI on non-crash episodes. The supervised piece anchors weights to sensible starting points; the RL piece fine-tunes them. *No tautology* — the agent's predictions can be wrong in ways the loss actually catches.

Slide 7 • DynamicWeightAgent — under the hood · 55s

[visual cue] Architecture flow diagram + training loss banner at bottom.

Architecturally, it's a small network. Two inputs — a four-dimensional kinematic vector and a five-dimensional context vector — get concatenated, run through two fully-connected ReLU layers of sixty-four hidden units each, then a softmax head that produces four weights summing to one.

The output isn't a category. It's the weight vector that goes into the closed-form AI score. So the AI is still computed in closed form — the agent is calibrating the weights, not replacing the formula.

The loss at the bottom is a convex combination: KL divergence against per-regime targets early in training, RL term later. *That schedule is what Professor Daher and I agreed to in the week-one sync.*

Slide 8 • The gating network · 55s

[visual cue] Gate math on left + per-regime weight convergence plot on right.

On top of the agent there's a gating network. The motivation: a hard switch on regime — if highway, use highway weights; if urban, use urban weights — produces unphysical discontinuities at boundaries. Traffic density doesn't jump in a single timestep, and the agent should respect the gradient.

The gate is a small 2-layer MLP that maps context to a softmax over three regimes — highway, urban, weather. The predicted weight vector is a convex combination of three learnable regime embeddings, weighted by the gate. One hundred forty-three trainable parameters total — a compositional layer, not a deep classifier.

The plot on the right: solid lines are the agent's predicted weights; dashed are the per-regime targets. The agent settles within 0.03 L1 across all three regimes, with a smooth twelve-step ramp at boundaries instead of a one-step snap.

Slide 9 • The ablation that matters · 35s

[visual cue] Two horizontal bars: 'WITH CONTEXT 0.05 KL' vs 'CONTEXT ZEROED 0.18 KL'. Bottom navy strap with conclusion.

Quick ablation, requested by Professor Daher: zero the context input, retrain. If the gate still works, the context features were never doing anything. If it doesn't, they're carrying the discriminative signal.

With context, KL loss is 0.05 — what we saw on the previous slide. Context zeroed: KL jumps to 0.18, and the regime embeddings collapse toward the highway target — the dominant regime in the training set. So the architecture isn't a glorified lookup table. The context features are doing real work.

Slide 10 • The Q-learner was hiding something · 60s

[visual cue] Two big stat cards: 'BELLMAN ERROR — 95% < 0.05' and 'STATE VISITATION — 70%'. Red strap at the bottom with the reading.

Second piece of work: diagnostics. Two instruments, both implemented in week five. First, log the Bellman error per Q-learner update — the temporal-difference residual. Second, build a state-visitation heatmap over the discretized state space.

What we found. Ninety-five percent of Bellman errors fall under 0.05 — looks great. But the right tail extends out to 0.6, and those errors concentrate in one corner: low speed, high proximity. Now look at the visitation heatmap on the right — seventy percent of all visitation is in the *opposite* corner: fast and unobstructed.

The reading. The Q-learner had implicitly learned to avoid exactly the under-explored states where its predictions would fail. The midterm's clean accuracy curve was an artifact of the agent choosing its own evaluation distribution. *That is the conservative-bias problem.*

Slide 11 • A count-based curiosity bonus · 50s

[visual cue] Equation banner at top + three big stat cards: -50%, +22pp, 70→40%.

The fix is intrinsic motivation. The formula at the top — total reward equals environment reward plus beta over the square root of the visit count plus one. Bellemare et al., 2016. Count-based, no learned predictor needed because the Q-learner is tabular.

After a beta sweep, beta of 0.1 is the operating point. Three results, left to right. Bellman error in the high-proximity column drops fifty percent. Per-class accuracy on the rare Conservative class jumps twenty-two percentage points. Easy-corner visitation drops from seventy to forty percent — *the agent is now actually exploring*.

Slide 12 · From highway-env to SUMO · 40s

[visual cue] Left side: embedded simulation video. Right side: SUMO intersection diagram + 3 stat boxes.

Third piece — the simulator change. Highway-env was great for solo-agent training but doesn't scale to population-level questions. *Does one aggressive driver create a measurable shockwave through a fleet of normals?* That needs hundreds of vehicles, realistic car-following, and a real-time traffic-control API. SUMO gives us all three.

On the left, the highway-env demo from last term. On the right, one of three SUMO scenarios — a four-arm signalized intersection. Up to a thousand vehicles per run, fifteen-Hertz TraCI stepping.

Slide 13 · The heterogeneous-driver shockwave · 55s

[visual cue] Big '+6.8 AI' callout on left. Three spatial bins on right: CLOSE +12.4, MEDIUM +6.6, FAR +1.5.

Here's the central empirical finding of the term. Take a population of all-Normal drivers — that's the reference — and seed five percent of them as Aggressive. Re-run, compute per-vehicle mean AI for the Normals, compare against reference.

The Normals score six-point-eight points higher when five percent of the population is aggressive. *That's the shockwave.* Same vehicles, same scenario — just a small fraction of aggressive neighbors, and the rest of the population starts driving more aggressively too.

The decomposition on the right shows where the effect lives. Bin the Normals by minimum distance to any Aggressive driver. Close — within thirty meters — plus 12.4 above reference. Medium — plus 6.6. Far — only plus 1.5, essentially noise. *So the shockwave is local.* Vehicles that come into spatial proximity with an aggressive driver are the ones absorbing the effect.

Slide 14 · Small fractions do most of the damage · 45s

[visual cue] Left: line chart of population mean AI vs aggressive fraction (0 → 20%). Right: saturating-fit equation, $R^2 = 0.998$, conclusion.

Now sweep the aggressive fraction from zero to twenty percent. Three random seeds per condition. The chart on the left is the result.

The marginal effect drops sharply. Going from zero to five percent is a six-point-eight bump. Going from fifteen to twenty percent is only zero-point-eight. A saturating-exponential fit gives R-squared of 0.998. The saturation constant is six percent — meaning the curve reaches sixty-three percent of its asymptote at six percent aggressive drivers, ninety-five percent at eighteen percent.

The policy-relevance reading: *single-digit percentages of aggressive drivers cause most of the population-level damage.* That's also the regime where targeted mitigation has the highest payoff — which leads to the next slide.

Slide 15 · Social Latency: a reward with accountability · 60s

[visual cue] Equation banner at top + λ -sweep table on left + 75% fleet result on right.

The mitigation. The midterm reward — minimize your own AI — produces paralysis when an aggressive neighbor pulls up: brakes, big gap, the very shockwave we just measured.

So the new reward at the top adds a Social Latency penalty. Total reward equals one-minus-AI-self, minus lambda times the maximum of zero and the increase in mean AI among neighbors within thirty meters.

The clip at zero matters — we penalize only increases the agent causes.

In the lambda sweep, lambda equals two is the Pareto pick. Self-AI rises only 2.2 points; neighbor shockwave drops forty percent.

The fleet result on the right is the headline. Replace fifty percent of Normals with lambda-two agents in the five-percent-aggressive scenario. *Seventy-five percent of the shockwave damage recovered.* Cost: 2.8 percent more average trip time. *A remarkably favorable trade.*

Slide 16 · A weather regime, derived from observables · 45s

[visual cue] Left: slip estimator equation + per-channel correlations. Right: regime-mixture bars + low-friction shockwave delta.

Friction is harder than it looks. SUMO doesn't expose slip ratio directly, so we estimate it from observables we do have.

The estimator on the left: slip is approximately the gap between actual velocity and the IDM equilibrium velocity, normalized. On dry roads, near zero. On wet roads with friction 0.4, slip rises to 0.15–0.25 during accel events. We add it to the gate's context vector.

The bars on the right show the regime mixture in the wet scenario — the gate flips to about fifty percent weather, exactly the response we want without hand-coding. As a bonus, the shockwave is 7.6 wet versus 6.8 dry — *aggressive drivers are slightly more impactful on slippery roads*, matching basic vehicle-dynamics intuition.

Slide 17 · ROS2 wrapper — the integration surface · 40s

[visual cue] Three-node flow diagram + four latency stat boxes.

So we have an agent, math, results. Now to plug it into something. That's the ROS-2 wrapper.

Three nodes. TelemetryPublisher reads SUMO logs and publishes at fifteen Hertz. AgentNode in the middle subscribes, runs the gate-head agent in inference mode, publishes on the env-aggressiveness-index topic — that's what Project 8, the orchestrator project, subscribes to.

Latency budget: median eighteen milliseconds, ninety-ninth percentile forty-one — both under the fifty-millisecond budget the orchestrator team set. And bag-record / replay produces bit-identical output. *The system is reproducible end-to-end.*

Slide 18 • 100% accuracy was a bug · 55s

[visual cue] Title in red. Two side-by-side architecture cards: BEFORE (info leak, red) → AFTER (separated paths, lime).

Final piece. Eight-day integration push, May second through ninth. This slide is the bug; the next is the fix.

May second was supposed to be buffer. Final smoke test: agent versus ground truth was *one hundred percent* across all three scenarios. *That's a textbook bug signal.* Perfect agreement between supposedly independent systems means they're not.

The diagram on the left shows the leak. Both paths were calling the same TraCI methods on the same vehicles in the same step. They were computing the same thing.

The fix on the right is architectural. Separate the assessors. SumoGroundTruth reads exact values; SumoAgentAssessor adds Gaussian sensor noise — calibrated to onboard radar and lidar — *before* running the formula. Now we're testing whether the formula survives sensor noise on independent code paths.

Slide 19 • What the leak was hiding · 40s

[visual cue] Four numbered issue cards: Squared speed, Proximity to ego, Double-counted weather, Two-lane intersection.

Once the architectural separation was in place, three more bugs surfaced — they had been agreeing with themselves the whole time. Quick walkthrough.

One: the speed term was squared. With the 0.30 weight, speed contribution capped at thirty percent — below the fifty-five percent Aggressive threshold. Fix: linear, with per-scenario reference speeds.

Two: proximity was reading distance to the ego vehicle, not to each NPC's actual leader. Three: weather had a double-counted speed reduction, capping aggressive weather drivers at nine meters per second — crawling. Four: urban arms had two lanes, so aggressives lane-changed past conservatives. Switched to one-lane arms with explicit conservative-aggressive platoon pairs.

Slide 20 • Calibrated. Independent. Validated. · 50s

[visual cue] Three big % stat cards: 91.0% Highway, 81.6% Urban, 88.3% Weather. Lessons banner at the bottom.

Calibration locked in May eighth. Final agent-versus-ground-truth accuracies, with realistic Gaussian sensor noise on the agent's input but not on the ground truth.

Highway: ninety-one percent across 1,328 samples. Urban: eighty-one-point-six percent across 2,284. Weather: eighty-eight-point-three percent across 1,825. All three scenarios produce all three driver categories. *No info leak.* This is a real validation.

Urban falls below the ninety-to-ninety-five percent target — intersection speeds put Normal and Aggressive populations close to the category boundary, so sensor noise more easily flips classifications. The mechanism is well-understood, not architectural.

And the lesson, twice. *One hundred percent on anything is a bug signal.* Both times this term, the fix was infrastructure, not architecture.

Slide 21 · What we built — and where it goes · 45s

[visual cue] Left: list of 14 artifacts shipped. Right (navy): five future-work items.

Wrapping up. Fourteen artifacts shipped this term, listed on the left: the agent and gate, the diagnostic suite and curiosity bonus, the SUMO wrapper, the population experiments, the friction-aware regime, the ROS-2 stack, and the noise-validated assessor split.

Five future-work tracks on the right. CARLA integration, deferred this term. Learnable normalization saturation points. Correlated time-domain noise — the i.i.d. Gaussian model is a starting point but real onboard noise has temporal structure. Multi-agent RL with shared Social Latency objectives. And on-vehicle batched inference for the fifty-vehicle latency limit.

Submission goes up tomorrow. The codebase is tagged term-2-final-v2-with-noise-validation as of yesterday — that's the version that produced every figure and every number you've just seen.

Slide 22 · Q&A · 10s

[visual cue] Big 'Q & A' on dark slide. GitHub URL at the bottom.

Thank you. The repository link is at the bottom of the slide. I'm happy to take questions.